

COMICCRAFT AI

Eight Project Phases: Explanation & Separate Sample Code

Project: AI-assisted comic story creation

Prepared as a student project reference for use in VS Code.

How to use this document

Each phase includes a short explanation, a separate code/example artifact, and its expected output. The examples are educational and can be adapted to your actual application. The brainstorming, requirements, design, planning, documentation, and demonstration phases are represented by small scripts or project artifacts because these activities are not normally separate executable software modules.

Note: The development and testing examples share the file name `comiccrafft_demo.py`. Save that file first, then save the test file beside it.

1. Brainstorming & Ideation Phase

Explanation

In this phase, the project idea is identified and different features are discussed. For ComicCraft AI, the main idea is to help users turn a simple story idea into a short comic. Possible features include entering a prompt, creating a story outline, splitting it into panels, and showing dialogue. The team selects features that are useful and achievable.

Separate sample code / artifact

```
# brainstorming.py
project_idea = "ComicCraft AI"

ideas = [
    "Accept a story idea from the user",
    "Create a short story outline",
    "Divide the story into comic panels",
    "Add narration and character dialogue",
    "Display the comic in a readable format"
]

print("Project:", project_idea)
print("Selected features:")
for number, idea in enumerate(ideas, start=1):
    print(f"{number}. {idea}")
```

Expected output / deliverable: Output: A clear project concept and a list of proposed features.

2. Requirement Analysis Phase

Explanation

The team identifies what the application must do and records the requirements. Functional requirements describe features, while non-functional requirements describe qualities such as usability and response time. These requirements guide design and development.

Separate sample code / artifact

```
# requirements.py
functional_requirements = [
    "The user can enter a story prompt",
    "The system creates a story outline",
    "The system divides the story into panels",
    "Each panel can contain narration or dialogue"
]

non_functional_requirements = [
    "The interface should be easy to understand",
    "Results should be displayed clearly",
    "Invalid or empty prompts should be handled"
]

print("Functional requirements:")
for item in functional_requirements:
    print("-", item)

print("\nNon-functional requirements:")
for item in non_functional_requirements:
    print("-", item)
```

Expected output / deliverable: Output: A simple Software Requirements Specification (SRS) checklist.

3. Project Design Phase

Explanation

In the design phase, the application structure and user flow are planned before coding. A simple flow is: enter prompt → generate story → split into panels → display comic. The panel data structure defines how each panel stores its title, narration, and dialogue.

Separate sample code / artifact

```
# design.py
comic_panel = {
    "panel_number": 1,
    "scene": "A student discovers a mysterious book",
    "narration": "One quiet evening, Mira found an old book.",
    "dialogue": "Mira: What is this?"
}

screen_flow = [
    "Home / Prompt Input",
    "Generate Story",
    "Comic Panel Preview",
    "Save or Present"
]

print("Panel structure:")
print(comic_panel)
print("\nScreen flow:")
print(" -> ".join(screen_flow))
```

Expected output / deliverable: Output: A planned screen flow and data structure for comic panels.

4. Project Planning Phase

Explanation

The project is divided into manageable tasks with an order and estimated duration. Planning also identifies the tools needed, such as VS Code, Python or JavaScript, and any AI service used by the application. The schedule can be adjusted based on the project deadline.

Separate sample code / artifact

```
# planning.py
tasks = [
    ("Finalize idea and requirements", 1),
    ("Design interface and panel format", 1),
    ("Develop core comic workflow", 3),
    ("Test and fix issues", 2),
    ("Prepare report and demo", 1)
]

total_days = sum(days for _, days in tasks)

for task, days in tasks:
    print(f"{task}: {days} day(s)")

print(f"Estimated total: {total_days} days")
```

Expected output / deliverable: Output: A basic task plan and estimated timeline.

5. Project Development Phase

Explanation

This is the implementation stage. The planned features are converted into working code. The example below demonstrates a basic comic-panel generator from a user-provided story. It is a small educational prototype; connecting a real AI model or image-generation service would require the relevant API and configuration.

Separate sample code / artifact

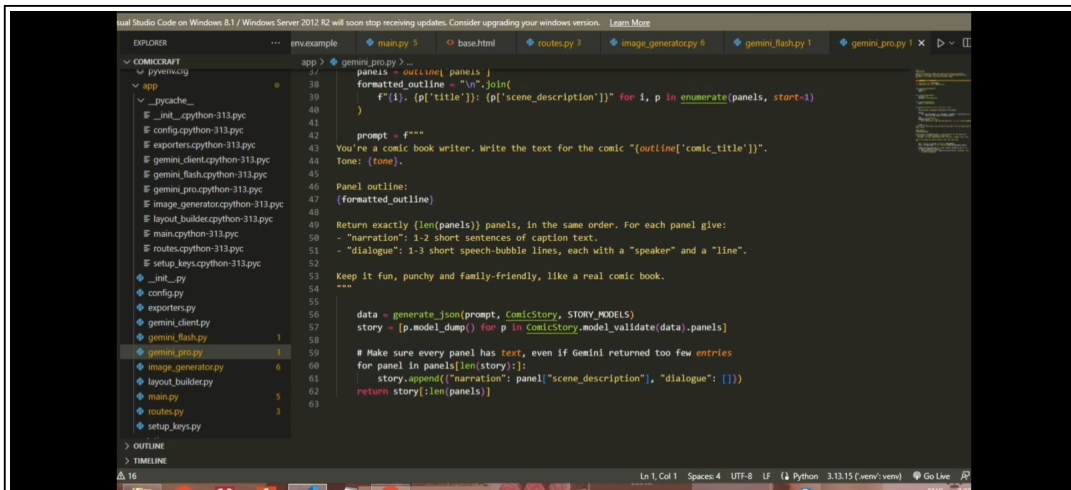
```
# comiccraft_demo.py
def create_comic(prompt):
    if not prompt.strip():
        return ["Please enter a story idea."]

    panels = [
        f"Panel 1 – Setup: {prompt}",
        "Panel 2 – Conflict: The main character faces a problem.",
        "Panel 3 – Action: The character tries a solution.",
        "Panel 4 – Ending: The story reaches a conclusion."
    ]
    return panels

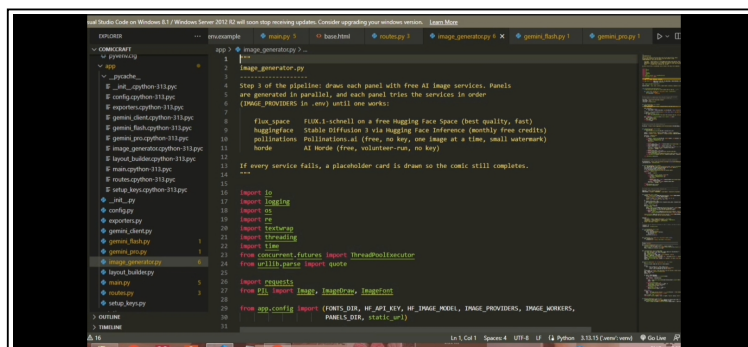
story_prompt = input("Enter your comic idea: ")
for panel in create_comic(story_prompt):
    print(panel)
```

Expected output / deliverable: Output: A working text-based prototype that converts an idea into a panel outline.

Actual Project Development Screenshots



Gemini story-generation code



AI image-generation code

6. Project Testing Phase

Explanation

Testing checks whether the application behaves as expected. Test normal input, empty input, and input containing only spaces. The sample uses Python's unittest framework to verify the core function. Bugs found during testing should be corrected and tested again.

Separate sample code / artifact

```
# test_comiccraft.py
import unittest
from comiccraft_demo import create_comic

class ComicCraftTests(unittest.TestCase):
    def test_valid_prompt_returns_four_panels(self):
        result = create_comic("A robot learns to paint")
        self.assertEqual(len(result), 4)

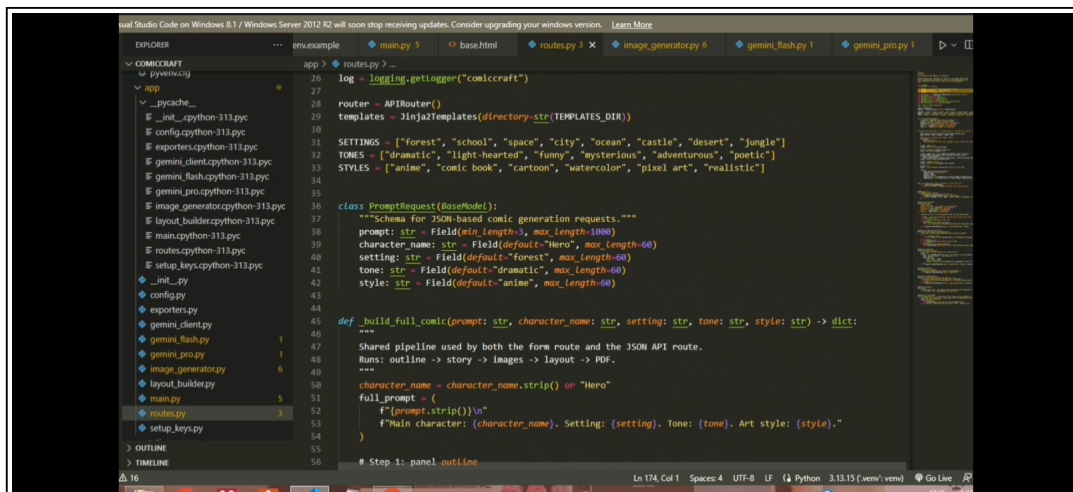
    def test_empty_prompt_shows_message(self):
        result = create_comic("")
        self.assertEqual(result, ["Please enter a story idea."])

    def test_whitespace_prompt_is_rejected(self):
        result = create_comic(" ")
        self.assertEqual(result, ["Please enter a story idea."])

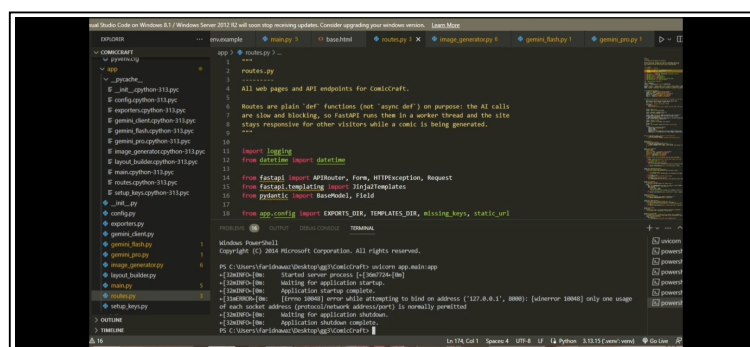
if __name__ == "__main__":
    unittest.main()
```

Expected output / deliverable: Output: Test results showing whether key functions pass or fail. Run with: `python -m unittest test_comiccraft.py`

Actual Project Testing / Execution Screenshots



ComicCraft route and API code



Application run / terminal output

7. Project Documentation Phase

Explanation

Documentation explains the purpose of the project, its features, setup steps, usage, and limitations. It helps another person understand and run the application. Keep the documentation consistent with the actual code and tools used in your project.

Separate sample code / artifact

```
# README.md
# ComicCraft AI

## Description
ComicCraft AI helps users turn a story idea into a comic-panel outline.

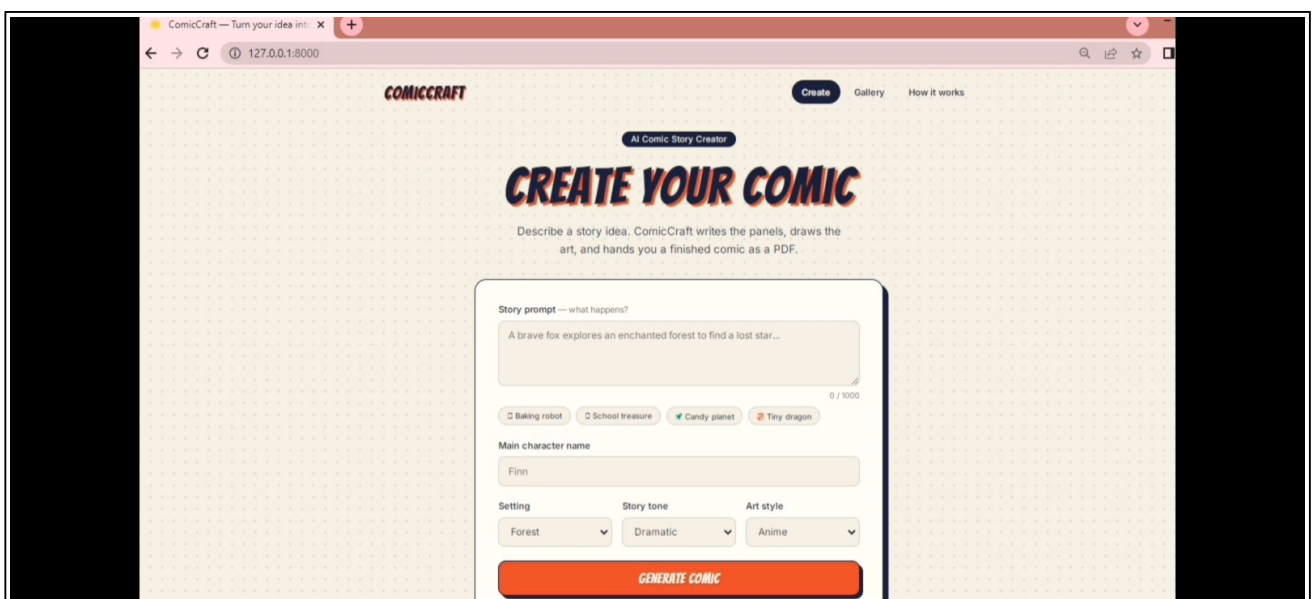
## Main Features
- Accepts a story prompt
- Produces a simple sequence of panels
- Displays a basic narration for each panel

## Run
1. Install Python 3.
2. Open the project folder in VS Code.
3. Run: python comiccraft_demo.py

## Note
This prototype creates text panels. AI-generated images require
integration with an image-generation service.
```

Expected output / deliverable: Output: README / user guide describing setup, features, and limitations.

Actual Project Interface Screenshot



ComicCraft web interface

8. Project Demonstration Phase

Explanation

In the demonstration phase, the completed prototype is presented to the teacher or audience. Explain the problem, show the input prompt, run the application, display the generated panels, and summarize testing and future improvements. The demo should use a prepared example and run smoothly.

Separate sample code / artifact

```
# demo.py
from comiccraft_demo import create_comic

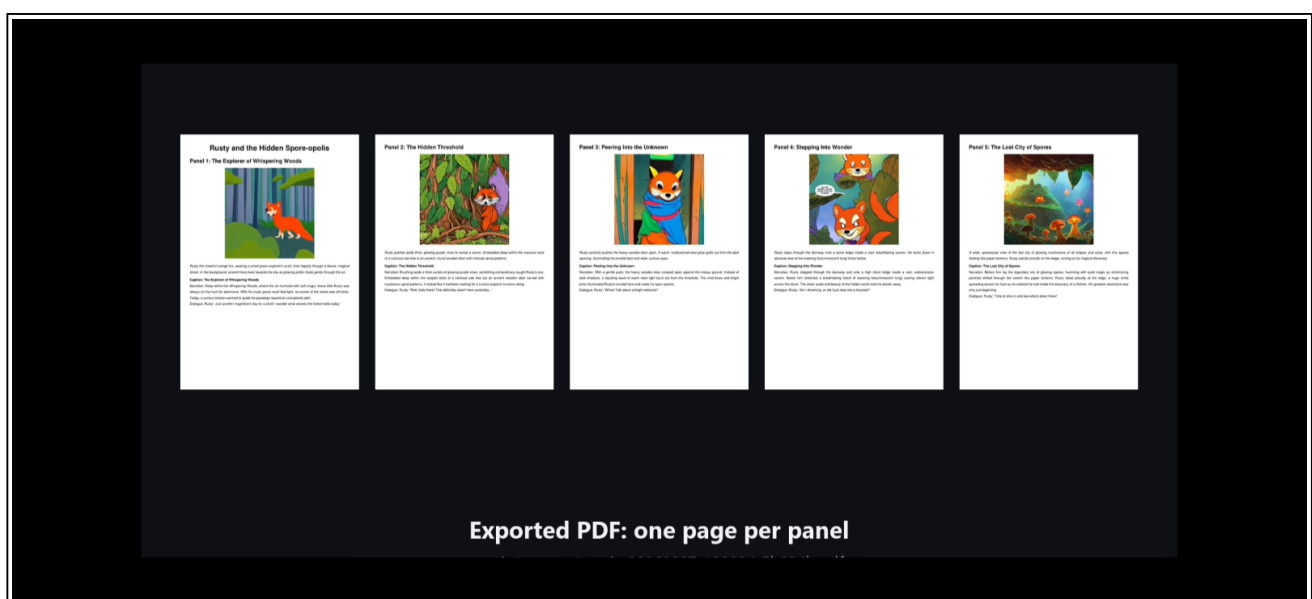
demo_prompt = "A young inventor builds a helpful robot"

print("COMICCRAFT AI — PROJECT DEMO")
print("Input idea:", demo_prompt)
print("\nGenerated comic:")
for panel in create_comic(demo_prompt):
    print(panel)

print("\nDemo completed.")
```

Expected output / deliverable: Output: A repeatable demonstration of the project's main workflow.

Actual Project Demonstration / Output



Generated comic exported as PDF